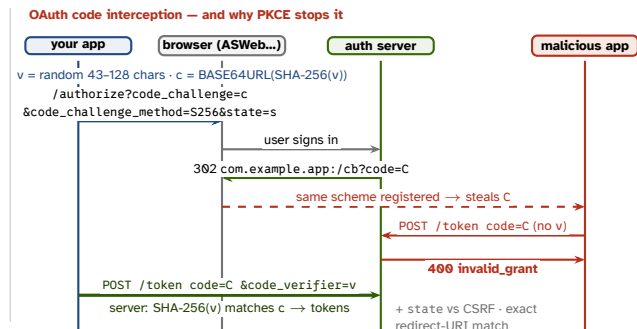
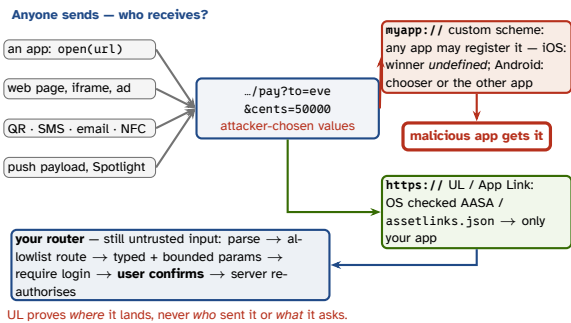


In one line: A deep link is a request **from a stranger**: any app, web page, QR code, SMS or push can fire it with any parameters. A **custom scheme** proves nothing — another app can register it and *receive* your links. A **Universal Link / App Link** proves the *destination* (your domain names your app), never the *sender*. So: parse and allowlist every parameter, confirm every action, carry no secrets — and protect OAuth redirects with **PKCE**.



How it works

- Custom scheme** (CFBundleURLTypes): no ownership. Apple: if several apps register a scheme, *which one opens is undefined*. Android: a custom-scheme intent filter anyone can copy → disambiguation dialog or the wrong app.
- Verified links:** iOS AASA (+ Associated Domains entitlement); Android `<intent-filter android:autoVerify="true"> + /.well-known/assetlinks.json` (delegate_permission/common.handle_all_urls, SHA-256 of the *Play App Signing* cert, not the upload key). Android 12+: an unverified web link opens the browser. Test: `adb shell pm get-app-links <pkg>`.
- Who called?** `sourceApplication` = the caller's bundle ID only when it is *your team*; another team → `nil`. A Universal Link carries no sender. You cannot authenticate the caller — authenticate the *user* and the *action*.
- Validate like an API endpoint:** `URLComponents`; exact scheme + host; allowlisted paths → a typed `Route`; bounded values; unknown → ignore / web. Never forward a `?next= / ?url=` into a `WebView` or `open(_:)` (open redirect, phishing, `WebView XSS`); never a file name into a path (traversal).
- State-changing links** (`/pay`, `/follow`, `/logout`, `/delete`) are CSRF for apps: show a confirmation screen with the real details; the server re-checks authorization. Hold the route until login completes.
- No secrets in links:** URLs land in history, logs, analytics, referrers, screenshots, clipboard, notification previews. **Magic link:** single-use, minutes-long TTL, bound to the device/session that asked, exchanged over TLS for a session — the link is a claim ticket, not the session.
- OAuth for native apps** (RFC 8252): system browser (`ASWebAuthenticationSession`), never an embedded `WebView`; redirect = claimed https, reverse-DNS scheme `com.example.app:/cb`, or loopback. **PKCE** (RFC 7636) is a MUST for public clients (RFC 9700, BCP 240, Jan 2025) + exact redirect-URI matching + no implicit grant. iOS 17.4: `CallbackURLScheme`.
- Android extras:** an exported activity is an entry point — treat `intent.data` and `extras` as input; never re-launch an Intent taken from `extras` (intent redirection → your unexported components).

Example — one router, every entry point

```
enum Route { case product(Int), pay(to: String, cents: Int) }
func route(_ url: URL) -> Route? {
  guard let c = URLComponents(url: url, resolvingAgainstBaseURL: false),
        c.scheme == "https", c.host == "example.com" else { return nil }
  let q = Dictionary((c.queryItems ?? []).map { ($0.name, $0.value ?? "") },
                    uniquingKeysWith: { first, _ in first }) // dup keys
  let p = c.path.split(separator: "/").map(String.init)
  switch p.first {
  case "p"? where p.count == 2: return Int(p[1]).map(Route.product)
  case "pay"? where p.count == 1:
    guard let to = q["to"], to.count <= 64,
          let cents = Int(q["cents"] ?? "") < (1...100_000).contains(cents)
    else { return nil }
    return .pay(to: to, cents: cents) // UI confirms; server re-authorises
  default: return nil // unknown: web page, never guess
  }
```

Attack → defence

scheme hijack	verified https links; nothing secret in the URL
OAuth code theft	PKCE S256 + state + system browser
action by link (CSRF)	confirmation screen + server authZ
open redirect	allowlist hosts; never forward <code>?url=</code>
crash by crafted link	total parser, no !, fuzz the router
token in link leaks	single-use, short TTL, device-bound
phishing lookalike	show the real payee / amount in-app

Traps

- ! "It's a Universal Link, so it's trusted" — the domain vouches for the app, not for the parameters.
- ! `Dictionary(uniqueKeysWithValues:)` on query items traps on `?to=a&to=b` — one crafted link crashes the app. Use `uniquingKeysWith`.
- ! Using `sourceApplication` as authentication — `nil` for every other team.
- ! Password-reset or session tokens in `myapp://` links — hijackable and logged.
- ! PKCE with plain — the challenge is the verifier; use S256.

Remember

"Stranger's request: parse, allowlist, confirm — carry nothing secret." UL = right door; PKCE = stolen key is useless.